

DOCKER - DEEP HINGLISH HANDBOOK

Images, layers, run modes, env variables, ports, networks, troubleshooting, logs, exec, and your MongoDB + mongo-express setup

**20 / 18 / 16 / 14 pt
typography**

1.7 line spacing

**Colourful visual
notes**

Golden rule: Docker commands ko ratne ke bajay 4 cheezein samjho: image kya hai, container ka main process kya hai, network kaunse containers ko jod raha hai, aur port host se container tak kaise aa raha hai.

This guide uses Linux-container concepts on Docker Desktop / Windows examples, plus MongoDB + mongo-express as a practical case.

1. Docker ko ek mental model se samjho

Docker ko ek mini-computer samjho jo tumhare host OS ke andar isolated processes chalata hai. Image ek read-only blueprint hai; container us blueprint ka running instance hai. Container ke andar jo process run hota hai, wahi container ki life decide karta hai.

Core relationship

Dockerfile	Image immutable layers	Container running process	Host / Internet
------------	------------------------------	---------------------------------	-----------------

Dockerfile se image build -> image se container run -> container host resources use karta hai; container ko network/port/volume chahiye ho to explicitly configure karte ho.

Windows par Docker Desktop generally Linux containers ko ek lightweight Linux VM/WSL2 environment ke through run karta hai. Isliye tum Windows PowerShell se command chalte ho, lekin container ke andar Linux commands jaise ls, sh, bash mil sakte hain.

Important: Docker image aur Docker network different concepts hain. Image download karna registry se content lana hai. Network create karna containers ko ek doosre se communicate karwana hai.

2. Docker Image me layers / levels kya hote hain?

Docker image me koi fixed universal number of "levels" nahi hote. Image ek ordered stack of filesystem layers hoti hai. Dockerfile ki kai instructions filesystem-changing layers create kar sakti hain; kuch instructions mainly metadata/config set karti hain. Base image khud bhi multiple parent layers se bani ho sakti hai.

Layer stack example

Layer 4 COPY app	Layer 3 RUN npm ci	Layer 2 COPY package*.json	Layer 1 Node base image
-----------------------------------	-------------------------------------	---	--

Suppose Dockerfile me:

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 5000
CMD ["node", "index.js"]
```

Practical idea: package files ko pehle copy karke npm ci ko separate build step rakhte hain. Jab sirf application source change hota hai, dependency layer cache ho sakti hai. Isse rebuild faster ho sakta hai.

Concept	Simple meaning	Benefit
Layer	Image filesystem ka immutable snapshot / delta	Reuse + caching
Layer cache	Same inputs ka existing build result	Build fast
Parent/base layers	Base image ke inherited layers	Common software reuse
Container writable layer	Running container ka top writable layer	Runtime changes isolate
Image ID / digest	Image content ki identity	Version/integrity tracking

Interview line: "Docker image is composed of layered filesystem changes. Layers are immutable and can be reused; a running container adds a thin writable layer on top."

3. docker run ke modes: -it, -d aur related flags

docker run ek naya container create karta hai aur usme image ka default command/entrypoint start karta hai. Flags decide karte hain container user ke terminal se kaise interact karega.

Command / flag	Meaning	Kab use kare?
docker run IMAGE	Foreground/default mode	Short-lived command ya logs dekhne ke liye
-i	STDIN open rakho	Interactive input chahiye
-t	Pseudo-TTY allocate karo	Terminal jaisa experience
-it	-i + -t	Shell/interactive apps
-d	Detached; background me run	Server / DB / long-running service
--rm	Stop hote hi container remove	Temporary testing
--name api	Human-readable name	exec/logs/stop easy
--restart unless-stopped	Container crash/restart policy	Services
--entrypoint	Default entrypoint override	Debug / custom process

Example:

```
docker run -it ubuntu bash

# detached web container
docker run -d --name nginx1 -p 8080:80 nginx

# temporary container
docker run --rm alpine echo "hello"
```

-it ko alag samjho: -i = input stream band mat karo; -t = terminal interface do. Dono milke interactive shell ke liye useful hain. -d me terminal attached nahi hota; Docker command tumhe container ID dekar prompt wapas kar deta hai.

Common confusion: "-d interactive mode" nahi hai. -it interactive terminal hai; -d detached/background style hai.

4. Container environment variables: -e kyun?

Environment variable container ke process ko configuration dene ka standard tarika hai. Code ko har machine par change karne ke bajay values outside the image inject kar sakte ho.

Variable	Example	Purpose
PORT	5000	App kis internal port par listen kare
NODE_ENV	production	Runtime behaviour/config
MONGO_URI	mongodb://...	DB location/credentials
ME_CONFIG_MONGODB_URL	mongodb://...	mongo-express ko MongoDB ka address
USERNAME/PASSWORD	root / secret	Authentication config

Set karne ke common ways:

```
docker run -e PORT=5000 -e NODE_ENV=production myapp

# from a file
docker run --env-file .env myapp
```

Kya har image ke liye env set karni padti hai? No. Bilkul nahi.

Image/documentation me agar application ko runtime configuration chahiye, tab set karo. Nginx ko run karne ke liye tumhe apni custom MONGO_URI nahi chahiye. MongoDB ko bhi sirf wahi env chahiye jo image ki initialization config expect karti hai (jaise root username/password).

Env na do to kya hota hai? 4 possibilities: (1) app default value use kare, (2) optional setting skip kare, (3) app startup error de, (4) wrong default ke saath run ho jaye. Exact behaviour image/application par depend karta hai.

Security note: secrets ko Dockerfile me bake karna avoid karo. Runtime secrets, secret managers, Compose secrets, or platform-level secret stores better approaches hain.

5. Container "Exited" status kyun aata hai?

Container VM nahi hai jo automatically zinda rahe. Container ka lifecycle uske main process se tied hota hai. PID 1 process exit karega to container generally stop ho jayega, aur docker ps -a me EXITED status dikhega.

Container lifecycle

Created	Running (PID 1 alive)	Stopped / Exited
---------	--------------------------	------------------

Symptom	Likely reason	Check
Exited (0)	Command successfully complete ho gaya	docker logs NAME
Exited (1)	Application/runtime error	docker logs NAME
Exited (137)	Often SIGKILL / memory kill	docker inspect; host/container memory
Exited immediately	Wrong command / config / dependency	docker logs + inspect
Running but unusable	App started but wrong port/config	logs + inspect + port/network

```
docker ps
docker ps -a
docker logs mongo-express
docker inspect mongo-express
docker start mongo-express
# run a new debug shell from image:
docker run --rm -it --entrypoint sh mongo-express
```

Golden debugging sequence: ps -a -> logs -> inspect -> exec (only if running) -> network/port checks.

6. Port binding: host port vs container port

Container ka internal network namespace alag hota hai. Agar app container ke andar port 5000 par listen kar raha hai, host browser ko automatically localhost:5000 par nahi milta. Host se container tak traffic expose karne ke liye port publishing/binding karte hain.

Port publishing

Browser / Host localhost:8081	Host port 8081	Docker forwarding 8081 -> 8081	Container 8081
----------------------------------	-------------------	--------------------------------------	-------------------

-p HOST_PORT:CONTAINER_PORT ka matlab: host ke given port par aane wali traffic ko container ke given port par forward karo.

```
docker run -d --name mongo-express -p 8081:8081 mongo-express
```

```
# common Node example
```

```
docker run -d --name api -p 5000:5000 my-api
```

Term	Meaning
Host port	Tumhare Windows/Linux host par port; e.g. 8081
Container port	Container ke network namespace me app ka listening port; e.g. 8081
Published port	Docker ne host/container mapping create ki
EXPOSE 8081	Image metadata/docs: "app expects 8081"; by itself host port publish nahi karta
-p 8081:8081	Actual host-to-container publishing

Port binding na karo to? Same Docker network ke dusre containers phir bhi container port par service ko reach kar sakte hain (user-defined network me), lekin host/browser se direct access nahi milega, unless you use another mechanism such as host networking or a proxy/gateway.

Kitne host ports? TCP port range 0-65535 aur UDP bhi 0-65535 concepts hain. Lekin practical availability OS permissions, reserved ports,

already-used ports, firewall aur Docker limitations par depend karti hai. Tum ek host port ko same protocol/address combination par multiple containers ko generally same time publish nahi kar sakte.

Important: Container ke andar service ko usually 0.0.0.0:PORT par listen karna chahiye, sirf 127.0.0.1 par nahi. Warna published port ke baad bhi external traffic fail ho sakta hai.

7. Docker troubleshooting commands - kya, kyun, kab

Command	Kya karta hai	Kyun use karte ho
docker version	CLI + server info	Client/daemon connectivity/version check
docker info	Docker engine details	General environment diagnosis
docker ps	Running containers	Kaun chal raha?
docker ps -a	All containers	Exited containers dekhna
docker images	Local images list	Image available hai?
docker image ls	Same purpose	Images check
docker pull IMAGE	Registry se image download/update	Local image missing
docker run ...	Container create + start	App start
docker start NAME	Stopped container start	Existing container reuse
docker stop NAME	Graceful stop	Service shutdown
docker restart NAME	Stop + start	Quick recovery
docker rm NAME	Container delete	Cleanup
docker rmi IMAGE	Image delete	Disk cleanup
docker logs NAME	Container stdout/stderr	Application error/root cause
docker logs -f NAME	Logs follow	Live logs
docker inspect NAME	Low-level JSON metadata	Env, command, mounts, network, IP, ports
docker top NAME	Processes in container	PID/process diagnosis
docker stats	Live CPU/RAM/network/block I/O	Resource issue
docker port NAME	Published ports mapping	Host->container mapping check
docker diff NAME	Filesystem changes	Unexpected runtime file changes
docker events	Engine events stream	What Docker is doing live

Command	Kya karta hai	Kyun use karte ho
docker system df	Docker disk usage	Disk full diagnosis
docker system prune	Unused resource cleanup	Disk recovery; carefully

Production caution: docker system prune blindly mat chalao. Unused images/containers/networks/volumes delete ho sakte hain depending on flags. Pehle docker system df se inspect karo.

8. docker exec -it ID /bin/bash /bin/sh ko ek-ek word me samjho

```
docker exec -it 8733449d6c3a /bin/bash
```

Part	Meaning
docker	Docker CLI
exec	Already running container ke andar ek naya process execute karo
-i	STDIN open rakho
-t	Pseudo-TTY allocate karo
8733449d6c3a	Container ID (short ID bhi often works)
/bin/bash	Container filesystem me Bash shell ka executable path
/bin/sh	Portable POSIX-ish shell; lightweight images me often available

Why /bin/bash vs /bin/sh? Ubuntu/Debian based images me bash common hota hai. Alpine images minimal hote hain aur often bash installed nahi hota, lekin /bin/sh available hota hai. Isliye docker exec -it CONTAINER sh ek safer first attempt hai.

```
docker exec -it mongo-express sh

docker exec -it mongo-express /bin/sh

# run one command, shell open nahi karna:
docker exec mongo-express env
docker exec mongo-express ls -la /app
```

Key difference: docker exec sirf running container me chalega. Exited container ke liye docker start karo, ya image se fresh temporary debug container run karo.

9. Docker network kya hai aur kyun use karte hain?

Docker network container ko network namespace aur connectivity rules deta hai. Most important real-world use: ek application container ka DB container, cache container, backend, reverse proxy etc. se baat karna.

Two-container application

Browser Host	mongo-express app container	MongoDB database container
---------------------	------------------------------------	-----------------------------------

User-defined bridge network me container names Docker DNS ke through resolvable hote hain. Isi liye connection string me tarunmongo hostname kaam kar sakta hai - IP ya localhost likhne ki zarurat nahi.

Network type	Meaning / use
bridge	Default Linux bridge for containers; basic isolation/connectivity
user-defined bridge	Recommended for app stacks; built-in name-based DNS and better isolation
host	Container host network namespace share karta hai (Linux use-cases; Docker Desktop caveats)
none	No normal network connectivity
overlay	Multi-host container networking, commonly Swarm
macvlan	Container ko MAC-level network presence; specific infra use-cases
ipvlan	Alternative network driver for specific L2/L3 requirements

Rule of thumb: Most local multi-container projects ke liye user-defined bridge network enough hota hai. Example: tarun-network me MongoDB + mongo-express + backend.

10. Docker network commands - command ka matlab aur kyun

Command	Meaning / why
docker network ls	All Docker networks list
docker network create tarunmongo	New user-defined bridge network create
docker network inspect tarunmongo	Network details, driver, subnet, connected containers
docker network connect tarunmongo app	Existing running container ko network se attach
docker network disconnect tarunmongo app	Container ko network se detach
docker network rm tarunmongo	Unused network delete
docker network prune	Unused networks clean
docker run --network tarunmongo ...	Container start hote hi chosen network attach
docker run --network none ...	No normal network
docker run --network host ...	Host network mode where supported

Useful inspect workflow:

```
docker network ls
docker network inspect tarunmongo
docker inspect mongo
docker inspect mongo-express
```

Inspect output me Containers section ke andar names, IDs, IPv4Address etc. dekh sakte ho. Agar mongo-express same user-defined network par nahi hai, to tarunmongo hostname resolve nahi hoga.

Remember: docker network create network banata hai. --network NAME container ko us network par connect karta hai. Dono alag steps hain.

11. Kya image download karne ke liye Docker network zaroori hai?

Short answer: Docker container network nahi. docker pull mongo ke time Docker daemon registry (for example Docker Hub) se image layers fetch karta hai. Uske liye host/daemon ka Internet access, DNS, proxy/firewall etc. important hain. Tumhe pehle docker network create tarunmongo karne ki zarurat nahi.

Pull vs run networking

docker pull mongo	Registry / Internet	docker run --network tarunmongo	Container network
--------------------------	----------------------------	--	--------------------------

Action	Network concept
docker pull mongo	Registry connectivity / Internet. User-defined container network is irrelevant.
docker build .	Build may need network access if FROM or RUN needs registry/package downloads; network behaviour is build/engine dependent.
docker run mongo	Container gets a network from Docker default/config.
docker run --network tarunmongo mongo	Container explicitly joins tarunmongo.
mongo-express -> MongoDB	Same user-defined network gives service discovery by container name.

Toh "network use na kare" to? If by that you mean no custom user-defined Docker network: image can still be pulled. A container can still run using Docker default networking. But your containers may not have the same predictable name-based service discovery setup, so multi-container communication becomes harder/less explicit.

Very important distinction: "Internet/network required" != "Docker network named tarunmongo required". First is external connectivity; second is a Docker-level virtual network for containers.

12. Tumhara mongo-express command - line by line

```
docker run -d ^
-p 8081:8081 ^
--network tarunmongo ^
--name mongo-express ^
-e ME_CONFIG_MONGODB_ADMINUSERNAME=root ^
-e ME_CONFIG_MONGODB_ADMINPASSWORD=2805 ^
-e ME_CONFIG_MONGODB_URL=mongodb://root:2805@tarunmongo:27017/ ^
mongo-express
```

Part	Meaning
docker run	New container create + start
-d	Detached mode: terminal ko block nahi karega
^	Windows CMD/PowerShell-compatible line continuation in contexts where ^ is supported; PowerShell commonly uses backtick ` instead. One-line command avoids shell confusion.
-p 8081:8081	Host port 8081 -> container port 8081
--network tarunmongo	Join user-defined Docker network named tarunmongo
--name mongo-express	Container ka fixed easy name
-e ...USERNAME=root	Runtime environment variable
-e ...PASSWORD=2805	Runtime environment variable
-e ...URL=mongodb://...	MongoDB connection string; hostname tarunmongo
mongo-express	Image name

Sabse critical line:

```
ME_CONFIG_MONGODB_URL=mongodb://root:2805@tarunmongo:27017/
```

Yahan tarunmongo ka matlab "MongoDB container ka hostname/name" ho sakta hai, provided Mongo container isi user-defined network par connected ho aur uska name tarunmongo ho. 27017 MongoDB ka container-side port hai. Yahan localhost use karna wrong hoga, kyunki mongo-express ke container ke liye localhost matlab mongo-express khud.

Concept shortcut: Browser -> host:8081 -> mongo-express:8081.
Mongo-express -> tarunmongo:27017 -> MongoDB. Host port 8081 aur
MongoDB container port 27017 alag traffic paths hain.

13. Documentation me exact command nahi mila - kya sab yaad karna hai?

Nahi. Docker ecosystem bahut bada hai. Tumhe hundreds of commands yaad nahi karne. Tumhe flag ka category aur help/documentation se exact image-specific variable nikalna aana chahiye.

Category	Core things to remember
Lifecycle	ps, ps -a, run, start, stop, restart, rm
Debug	logs, inspect, exec, top, stats
Images	pull, images, rmi, history
Ports	-p HOST:CONTAINER, docker port
Env	-e KEY=VALUE, --env-file FILE
Networks	network ls/create/inspect/connect/disconnect/rm
Storage	-v / --mount
Cleanup	system df, prune

Image-specific flags kaise discover kare?

```
docker pull mongo-express
docker image inspect mongo-express
# official docs / Docker Hub page
# run help when image supports it
docker run --rm mongo-express --help
```

Application image ke env variable names Docker ke generic commands nahi hote. ME_CONFIG_MONGODB_... mongo-express application ke own configuration knobs hain. Isliye Docker documentation me nahi, image/application documentation me milenge.

Interview + real project strategy: Docker ke generic 15-20 commands strong karo; image-specific env/ports/entrypoint ko documentation se lookup karna normal professional practice hai.

14. MongoDB + mongo-express ka correct conceptual setup

Assume tumne MongoDB container ka naam tarunmongo rakha hai aur custom network bhi tarunmongo naam ka banaya hai. Name conflict avoid karne ke liye production-style naming me network ko tarun-net jaise naam dena cleaner hota hai. Example below:

```
docker network create tarun-net

docker run -d --name tarunmongo \
  --network tarun-net \
  -e MONGO_INITDB_ROOT_USERNAME=root \
  -e MONGO_INITDB_ROOT_PASSWORD=2805 \
  mongo

docker run -d --name mongo-express \
  --network tarun-net \
  -p 8081:8081 \
  -e ME_CONFIG_MONGODB_ADMINUSERNAME=root \
  -e ME_CONFIG_MONGODB_ADMINPASSWORD=2805 \
  -e ME_CONFIG_MONGODB_URL=mongodb://root:2805@tarunmongo:27017/ \
  mongo-express
```

Flow: both containers same tarun-net -> Docker DNS resolves tarunmongo -> mongo-express talks to MongoDB on 27017. Only mongo-express is published to host via 8081. MongoDB need not be published to the host unless you specifically need host-side Mongo tools.

Why not -p 27017:27017 for Mongo by default? Internal container-to-container traffic needs no host port binding. Avoid exposing DB to host/network unless you have a reason and firewall/authentication are configured.

15. Windows CMD / PowerShell vs Linux shell

Tumne command me ^ dekha. Ye Docker syntax nahi hai; ye shell line-continuation syntax hai. Isi liye copy/paste karte waqt shell ka difference samajhna important hai.

Shell	Typical multiline continuation
Windows CMD	^
PowerShell	Backtick `
Linux/macOS bash/zsh	\

```
:: CMD example
docker run -d ^
  -p 8081:8081 ^
  --name mongo-express ^
  mongo-express

# PowerShell example
docker run -d `
  -p 8081:8081 `
  --name mongo-express `
  mongo-express

# Linux/macOS bash
docker run -d \
  -p 8081:8081 \
  --name mongo-express \
  mongo-express
```

Safest beginner trick: pehle one-line command chalao. Jab command working ho jaye, tab multiline formatting seekho.

16. All-purpose Docker troubleshooting playbook

Case A - container immediately exits

```
docker ps -a
docker logs CONTAINER
docker inspect CONTAINER --format '{{json .Config.Cmd}}'
# if you need a shell, run a temporary debug container from the image
docker run --rm -it --entrypoint sh IMAGE
```

Case B - localhost:PORT open nahi ho raha

```
docker ps
docker port CONTAINER
docker logs CONTAINER
docker inspect CONTAINER
# inside running container:
docker exec -it CONTAINER sh
# then check app/listening process if tools exist
```

Case C - mongo-express cannot connect to MongoDB

```
docker ps -a
docker logs mongo-express
docker logs tarunmongo
docker network inspect tarun-net
docker inspect mongo-express
docker inspect tarunmongo
```

Case D - "ENOTFOUND tarunmongo"

Typically hostname resolution failure. Check: container name, network name, whether both containers are attached to the same user-defined network, and whether the hostname in the URL exactly matches the reachable service/container name.

Case E - "ECONNREFUSED 127.0.0.1:27017"

Application container me localhost use hua ho sakta hai. Container ke andar 127.0.0.1 means that same container, not MongoDB container. Use the Mongo service/container hostname on the shared network, e.g. tarunmongo:27017.

Remember: ENOTFOUND = name resolution issue; ECONNREFUSED = address resolved, but nothing accepted the connection at that address/port (or access was blocked).

17. One-page Docker command cheat sheet

```
docker --help
docker run --help
docker ps
docker ps -a
docker images
docker pull IMAGE
docker run -d --name NAME -p HOST:CONTAINER IMAGE
docker start NAME
docker stop NAME
docker restart NAME
docker rm NAME
docker logs NAME
docker logs -f NAME
docker exec -it NAME sh
docker inspect NAME
docker top NAME
docker stats
docker port NAME
docker network ls
docker network create NET
docker network inspect NET
docker network connect NET NAME
docker network disconnect NET NAME
docker network rm NET
docker network prune
docker system df
docker system prune
```

Think in this order:

Question	Command
Kya chal raha hai?	docker ps
Kya fail hua?	docker ps -a
Kyun fail hua?	docker logs NAME
Exactly config kya hai?	docker inspect NAME
Port kaha ja raha hai?	docker port NAME
Kaunse network me hai?	docker network inspect NET
Andar kya process hai?	docker top NAME
Resource issue?	docker stats
Andar shell chahiye?	docker exec -it NAME sh

18. Final memory map - kya yaad rakho, kya lookup karo

Must remember: image vs container, foreground vs detached, -it, -e, -p
HOST:CONTAINER, main process lifecycle, logs, inspect, exec, user-defined
bridge network, container-name DNS, and docker pull vs Docker network
distinction.

Lookup is normal: exact image-specific env variable names, special
entrypoints, image-specific ports, version-specific options, Compose
settings, cloud registry auth, complex BuildKit features.

Four questions

1. Image kya hai?	2. Process kya run kar raha?	3. Network me kiske saath baat?	4. Host se kaunsa port publish?
--------------------------	-------------------------------------	--	--

Your mongo-express command decoded in one sentence: "mongo-express
image se ek background container banao, host 8081 ko container 8081 se
publish karo, usse shared Docker network me join karo, fixed name do,
MongoDB credentials + connection URL env variables se do, aur MongoDB
ko network DNS name tarunmongo se reach karo."

Best learning approach: ek hi setup ko baar-baar todkar banao: network
create -> mongo run -> inspect -> mongo-express run -> logs -> exec -> port
check -> network inspect -> intentionally wrong hostname -> diagnose -> fix.
Isse concepts command se attach ho jayenge.

END OF HANDBOOK